Lab 07 — Netwerk: containers met elkaar laten praten

Theorie — hoe drie containers die elkaars adres niet kennen elkaar toch bij naam vinden, wat "een poort publiceren" echt betekent, en waarom `localhost` valkuil nummer één is.

Doelstellingen

- Begrijpen welk netwerk een container krijgt, en wat rootless daaraan verandert (`pasta`).
- Het standaardnetwerk onderscheiden van een **user-defined** netwerk, en weten waarom het tweede een DNS heeft.
- `-p`, `-P`, `EXPOSE`, `--network` en `--network-alias` kunnen lezen en schrijven.
- De drie klassieke fouten diagnosticeren: `localhost`, niet-gepubliceerde poort, naam die niet resolvet.
- De keten **web → api → db** op naam opbouwen, en het begrip *pod* begrijpen.

---

1. Het probleem: drie processen, drie eilanden

In lab 06 draaide `db` alleen. Zodra `api` en `web` erbij komen, moet nginx de API vinden en moet de API PostgreSQL vinden. Maar elke container is geïsoleerd: hij heeft zijn eigen netwerkstack, zijn eigen adres en **zijn eigen** **`localhost`**. In de container `api` wijst `localhost:5432` naar de API zelf, niet naar de database. Het is de vaakst gemaakte fout van alle beginners, en van heel wat gevorderden.

> **→ LINUX**
> Een **network namespace** geeft een proces zijn eigen netwerkstack: eigen interfaces, eigen routetabel, eigen firewallregels, eigen poorten. Twee namespaces kunnen elk een poort 8080 open hebben zonder conflict. `lo` (127.0.0.1) bestaat in elk van beide en leidt alleen naar zichzelf. Een container is onder meer een network namespace — daarom is `localhost` er een doodlopend straatje.

IP-adressen helpen evenmin: ze worden bij het aanmaken toegekend en veranderen wanneer een container opnieuw aangemaakt wordt. Een container-IP schrijf je nooit vast in een configuratie. Je hebt **namen** nodig.

2. Wat een container krijgt

Op een klassieke Docker maakt de root-daemon een **bridge** aan (`docker0`), sluit elke container erop aan via een paar virtuele interfaces, en doet NAT naar buiten. De container heeft een privé-IP (`172.17.0.x`) en gaat het internet op via het adres van de host.

> **→ NETWERK**
> Een **bridge** is een softwareswitch: interfaces die erop aangesloten zijn, vormen samen een lokaal netwerk. Een **veth**-paar is een virtuele kabel met het ene uiteinde in de container (`eth0`) en het andere op de bridge. **NAT** (*network address translation*) herschrijft het bronadres van uitgaande pakketten naar dat van de host, zodat de antwoorden terugkomen. Drie bouwstenen van de kernel, samengevoegd door de engine.

PODMAN
Een gewone gebruiker mag geen bridge aanmaken en geen NAT doen. In rootless vertrouwt Podman het standaardnetwerk daarom toe aan `pasta`, een programma in user space dat het adres van de host *kopieert* naar de container: `ip addr` in de container toont hetzelfde IP als je WSL, en `inspect` toont een leeg `IPAddress`. **Bridge**-netwerken blijven mogelijk (`--network podman`, of een netwerk dat je zelf aanmaakt): ze leven in een privé-namespace van Podman, en `rootlessport` geeft de gepubliceerde poorten door — en verbergt daarbij het adres van de echte client.

Netwerk	Hoe krijg je het	Adres van de container	DNS tussen containers
standaard (pasta)	niets opgeven	dat van de host, gekopieerd	nee
`podman` (bridge)	`--network podman`	`10.88.0.x`	**nee** (`dns_enabled: false`)
user-defined (bridge)	`podman network create stack` en dan `--network stack`	`10.89.x.x`	**ja** (`aardvark-dns`)
host	`--network host`	dat van de host, zonder isolatie	—
none	`--network none`	geen, alleen `lo`	—

3. Het user-defined netwerk en zijn DNS

Dit is de gouden regel van dit lab: **één applicatie = één netwerk dat je ervoor aanmaakt**. `podman network create stack` maakt een privébridge met een subnet (`10.89.0.0/24`) en vooral een **interne DNS-server** (`aardvark-dns`, op het adres van de gateway `10.89.0.1`). Elke container die met `--network stack` start, bereikt de andere **op hun containernaam**: `api` resolvet naar het IP van `api`, `db` naar dat van `db`. De `/etc/resolv.conf` van de container wijst naar `10.89.0.1`, met `search dns.podman`.

Docker werkt net zo: zijn standaardbridge heeft geen DNS, zijn user-defined netwerken wel. Het standaardnetwerk dient voor een snelle test; zodra twee containers met elkaar moeten praten, maak je een netwerk.

Een container kan tot **meerdere** netwerken behoren (`podman network connect admin api`) en **aliassen** dragen (`--network-alias backend`): de API is dan onder twee namen bereikbaar. Een netwerk isoleert ook: een container in het netwerk `admin` resolvet `api` niet zolang `api` er niet op aangesloten is.

VALKUIL
DNS resolvet een naam naar een IP **op het moment van de verbinding**. nginx daarentegen resolvet de namen in zijn `proxy_pass` **één keer, bij het opstarten**, en houdt het IP bij. Maak je `api` opnieuw aan en krijgt ze een ander adres, dan blijft nginx naar het oude sturen: `502 Bad Gateway`, `Host is unreachable` in zijn logs. Remedie: `web` herstarten, of nginx configureren met een `resolver`-directive en een variabele. Spring Boot heeft dat probleem niet: zijn connectiepool opent nieuwe verbindingen en resolvet dus opnieuw.

4. Een poort publiceren

In een bridge-netwerk is een container alleen bereikbaar voor zijn buren. Wil een client **van buiten** (je browser, `curl` vanuit WSL) hem bereiken, dan moet je een poort **publiceren**: `-p 18080:80` vraagt de engine om te luisteren op poort 18080 van de host en door te geven naar poort 80 van de container.

Syntaxis	Effect
`-p 18080:80`	host `0.0.0.0:18080` → container `80`
`-p 127.0.0.1:18080:80`	alleen vanaf de machine zelf
`-P`	publiceert alle `EXPOSE`-poorten op willekeurige hostpoorten (`39843`…)
`EXPOSE 8080` (Dockerfile)	**publiceert niets**: documentatie, gelezen door `-P` en door `podman ps`

`EXPOSE` is het klassieke misverstand: de image zegt "ik luister op 8080", meer niet. Zonder `-p` faalt `curl localhost:8080` vanaf de host, terwijl de containers van hetzelfde netwerk poort 8080 bereiken zonder enige publicatie. **Tussen containers publiceer je niet**; je publiceert voor de buitenwereld.

> **→ BEVEILIGING**
> Op Linux zijn de poorten **onder 1024** voorbehouden aan root. In rootless faalt `-p 80:80` : `rootlessport cannot expose privileged port 80 … bind: permission denied` (of `pasta failed` op het standaardnetwerk), en de container blijft in de toestand `Created`. Drie uitwegen: publiceren op een hoge poort (`-p 8080:80`), de grens verlagen (`sysctl net.ipv4.ip_unprivileged_port_start=80` , een beslissing van de beheerder), of er een reverse proxy voor zetten die als root draait. In het bedrijf luistert een container **nooit** rechtstreeks op 80 of 443: dat is de rol van de load balancer of de Ingress.

> **→ WINDOWS / WSL**
> Een poort die in WSL gepubliceerd is, bereik je vanuit Windows op `http://localhost:18080` : WSL 2 stuurt automatisch de poorten waarop de distributie luistert door naar `localhost` aan de Windows-kant — ook met `-p 127.0.0.1:…` . Andere machines op het netwerk zien die poort echter niet, tenzij je de *mirrored* netwerkmodus inschakelt (`.wslconfig` , `networkingMode=mirrored`) of een `netsh portproxy` -regel toevoegt.

5. host, none, en de host bereiken

`--network host` schaft de netwerkisolatie af: de container gebruikt rechtstreeks de stack van de host, zonder `-p` (dat heeft geen betekenis meer). Nuttig voor een netwerkdiagnosetool; gevaarlijk voor een dienst. En in rootless slaagt een nginx met `--network host` er niet in poort 80 te openen: `bind() to 0.0.0.0:80 failed (13: Permission denied)` .

`--network none` geeft alleen `lo` : geen enkele netwerktoegang, inkomend noch uitgaand. De juiste instelling voor een batchverwerking die met niemand hoeft te praten, of om te testen of een applicatie geen verborgen netwerkafhankelijkheid heeft.

Tot slot moet een container soms een dienst **van de host** bereiken (een database die in WSL geïnstalleerd is, een andere gepubliceerde container). De naam `host.containers.internal` wijst naar de host vanuit het standaardnetwerk — het equivalent van `host.docker.internal` bij Docker Desktop.

6. Pods

PODMAN
Een **pod** is een groep containers die **dezelfde network namespace** delen: hetzelfde IP, dezelfde poorten, dezelfde `localhost`. In een pod bereikt nginx de API op `localhost:8080`, en `-p` declareer je **op de pod**, niet op de containers. Het is precies het model van Kubernetes, waarvan Podman de woordenschat overneemt: `podman pod create --name stackpod -p 18090:80`, daarna `podman run --pod stackpod …`. Een onzichtbare *infra*-container houdt de namespace open. Docker heeft geen pods; `--network container:andere` is de benadering ervan.

Onthoud de nuance: in een **netwerk** praten containers met elkaar via hun naam; in een **pod** via `localhost`. Beide modellen bestaan naast elkaar, en het Kubernetes-deel van deze reeks bouwt op het tweede.

7. In het bedrijf

- **Eén netwerk per applicatie**, met drie diensten erop: `web`, `api`, `db`. Alleen `web` publiceert een poort. `db` wordt in productie **nooit** gepubliceerd; tijdens de ontwikkeling `-p 127.0.0.1:5432:5432` om er een SQL-client op aan te sluiten.
- **De API vindt de database op naam**: `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`. Een URL met `localhost` verraadt een configuratie van een werkstation die klakkeloos in een container gekopieerd is.
- **nginx is reverse proxy**: `/` serveert de frontend, `/api/` wordt doorgestuurd naar `api:8080`. De browser ziet maar één origin, wat CORS vermijdt, en de API staat niet rechtstreeks bloot.

→ HTTP
CORS (*cross-origin resource sharing*) is een regel van browsers: een pagina die door `localhost:18080` geserveerd wordt, mag `localhost:8080` niet aanroepen tenzij de API daar via headers mee instemt. Door frontend en API achter dezelfde nginx te zetten, op dezelfde origin, verdwijnt het probleem — een van de bestaansredenen van de reverse proxy, naast TLS en caching.

→ SPRING BOOT
De connectiepool (HikariCP) opent bij het starten enkele verbindingen en maakt er in de loop van de tijd nieuwe. Is `db` nog niet klaar, dan faalt Spring Boot bij het opstarten: `Connection refused`. Vandaar het belang van de opstartvolgorde en van *healthchecks* (lab 09), en van de optie `spring.datasource.hikari.initializationFailTimeout` om een database die wat later komt te tolereren.

Onthouden

- `localhost` in een container is de container zelf: andere containers spreek je aan op hun **naam**.
- Het standaardnetwerk (pasta in rootless) en het netwerk `podman` hebben geen DNS; een **user-defined** netwerk wel: één applicatie = één netwerk.
- `-p host:container` publiceert voor de buitenwereld; tussen containers van hetzelfde netwerk hoef je niets te publiceren. `EXPOSE` publiceert niets.
- In rootless geen poort < 1024: `-p 8080:80`, of een reverse proxy ervoor.
- nginx legt de IP's van zijn `proxy_pass` vast bij het opstarten: container opnieuw aangemaakt = `502`, nginx herstarten.

- `--network host` haalt de isolatie weg, `--network none` haalt het netwerk weg, `host.containers.internal` wijst naar de host.
- Een **pod** deelt de network namespace: `localhost` tussen containers, `-p` op de pod — het model van Kubernetes.

Woordenschat

network namespace: privé-netwerkstack van een container. — **bridge**: virtuele switch die containers verbindt. — **veth**: virtuele kabel container ↔ bridge. — **NAT**: adresherschrijving om via de host naar buiten te gaan. — **pasta**: standaard rootless-netwerk van Podman, kopieert het adres van de host. — **rootlessport**: doorgeefluik voor gepubliceerde poorten in rootless. — **aardvark-dns**: interne DNS van user-defined netwerken. — **publiceren (`-p`)**: een hostpoort doorgeven naar de container. — **`EXPOSE`**: documentaire declaratie van een poort. — **alias**: extra naam van een container op een netwerk. — **pod**: groep containers die de network namespace delen. — **reverse proxy**: server die verzoeken doorstuurt naar interne diensten.